

India · DPDP Act 2023 & Rules 2025

Citation-grounded RAG

Certinal

DPDP RAG Content Farm

Complete Project Documentation & Handover Guide

Phase 1 to present — the system, the RAG model, and how to run it

A single reference for anyone taking over this project — technical or non-technical.
It explains **what the system does**, **why every design choice was made**, **how the AI is kept honest**, and **how to operate and deploy it**.

Version 1.0 · 23 July 2026 · Prepared for handover

How to read this document

This guide serves two very different readers. You do not need to read all of it.

If you are...	Read these parts	You will understand
Non-technical (marketing, compliance, management)	Parts 1, 2, 3, then the plain-language intro of each later part	What the system produces, why it is safe to trust, the words used, and the overall shape
Technical, new to the project (engineer taking over)	All parts, in order. Part 5 (the RAG model) and Part 11 (operations) are the core.	Every component, the reasoning behind it, how to change it safely, and how to deploy
In a hurry	Part 1 (one page) + Part 4 (the architecture diagram) + Part 11 (runbook)	Enough to not break anything today

Throughout, sections are tagged **PLAIN** (written for everyone) or **TECHNICAL** (detail for engineers). Every diagram is followed by a **Legend** box explaining its symbols. Every number in this document is **measured**, not estimated.

The one idea that explains everything. This system has a single non-negotiable rule: *it must never state anything about the law that it cannot prove from the official text*. Almost every design decision in this document exists to enforce that rule. If you remember nothing else, remember that.

Table of contents

1	Executive summary — what this system is
2	The safety model — how the AI is kept honest
3	Glossary — every term in plain language
4	System architecture — the three surfaces and the shared engine
5	The RAG model, in depth — ingestion, retrieval, generation, verification
6	Phase-by-phase history — how the system was built (Phase 1 → now)
7	The content-style system — how the writing voice is controlled
8	The off-topic branch — handling non-DPDP requests safely
9	The n8n automation workflow — form to Google Doc
10	Evaluation & quality gates — the exam the system must pass
11	Operations & runbook — deploy, keys, keep-alives, troubleshooting
12	Repository map — where everything lives
A	Appendix — golden rules, effective dates, penalty slabs

1 • Executive summary PLAIN

What the system is, in one page.

The problem it solves. Certinal wants to publish marketing content — blogs, LinkedIn articles, newsletters — about India's new data-protection law, the **DPDP Act 2023** and the **DPDP Rules 2025**. Writing about law is dangerous: a confident sentence that is subtly wrong (a wrong deadline, a penalty that does not apply, an obligation quoted as if it were already in force) is a liability under the company's name. A general AI writing assistant will happily produce such sentences, because it writes from memory and memory is not a legal citation.

What this system does instead. It is a **citation-grounded content generator**. Before it writes a single sentence of law, it *retrieves* the exact passages of the Act and Rules that are relevant, and it is only allowed to write what those passages support — with a bracketed citation, like `[DPDP Act 2023, s.8]`, on every legal claim. A separate, automatic verifier then re-reads the draft and checks that every citation is real and every claim is supported. If it cannot verify the draft, the draft does not pass — it is corrected or sent to a human. Nothing is ever published automatically.

What it produces. A human operator (or an automated form) submits a topic and picks a format. The system returns a finished draft — correctly cited, correctly dated, carrying the mandatory "AI-assisted, not legal advice" disclaimer — as a Google Doc or on a web page, **for a human to review and approve before publishing**.

What it is	A Retrieval-Augmented Generation (RAG) system: it looks up the law, then writes strictly from what it looked up.
What it is NOT	Not a trained or fine-tuned model. Nothing here learns or updates model weights. It uses off-the-shelf AI models and controls them with grounding and verification.
The law it covers	DPDP Act 2023 (44 sections) + DPDP Rules 2025 (23 rules, 7 schedules) — 81 labelled pieces in total, and nothing else.
Formats it writes	Blog post, LinkedIn Article (Long & Short), Newsletter, plain prose.
Where it runs	Entirely in the cloud — a hosted API on Render, a web app on Streamlit, and an automation workflow on n8n. No dependency on any one person's laptop.
Current quality	Measured: 100% retrieval recall, 99% faithfulness, and 100% on every non-negotiable safety gate (see Part 10).

Why you can trust the output. Three independent safety nets stand between the AI and a published sentence: (1) it can only write from retrieved law; (2) an automatic verifier checks every citation; (3) a human must approve. A wrong sentence has to slip past all three — and the system is built so that the first two catch it, and the third is the backstop.

2 · The safety model — how the AI is kept honest

PLAIN

This is the heart of the project. Five rules, enforced in code — not left to the AI's goodwill.

The five golden rules

- 1. **Never write a legal claim the sources don't support.** If the retrieved law doesn't say it, the sentence is omitted. The AI is told, in its instructions, to never fill gaps from memory.
- 2. **If nothing relevant is found, refuse.** When a search for relevant law comes back empty or weak, the system outputs "no grounded source" rather than writing anything. It would rather say nothing than say something unproven.
- 3. **Cite the final law, never a draft.** The system quotes the *final notified* Rules (Gazette G.S.R. 846(E), 13 November 2025), never the earlier January 2025 draft.
- 4. **Never imply a future obligation is in force today.** Most of the Rules only take effect on 13 May 2027. Any draft that mentions such a duty must state its future date in plain words.
- 5. **Every published piece carries the disclaimer.** "AI-assisted, general information only, not legal advice." Added automatically, every time.

How the rules are enforced — the three gates

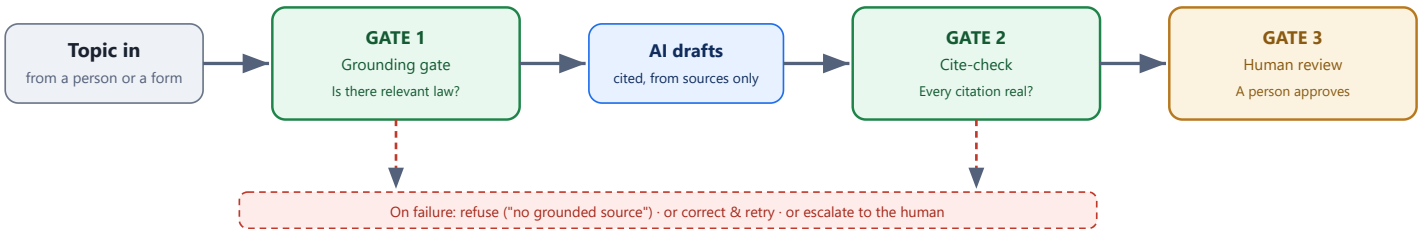


Diagram 2.1 — The three gates every piece of content must pass.

LEGEND — DIAGRAM 2.1

- green** An automatic safety gate (enforced by code, no human involved).
- blue** The AI writing step.
- amber** A step where a human is in control.
- grey** Input/output.
- solid** Normal flow of a draft moving forward.
- dashed red** Failure path — the draft is refused, corrected, or escalated, never silently passed.

Important nuance about "refusal." A refusal is the system *working*, not failing. When it says "no grounded source," it is correctly declining to invent law. When cite-check escalates a draft to a human, it is correctly flagging that it could not fully verify the draft on its own. Do not treat these as bugs to be "fixed" by loosening the checks.

3 · Glossary — every term in plain language PLAIN

Keep this page open while reading the rest. No prior knowledge assumed.

Term	Plain meaning
DPDP Act 2023 / Rules 2025	India's data-protection law and its detailed rules — the <i>only</i> things this system is allowed to quote from.
RAG (Retrieval-Augmented Generation)	The technique at the core of the system: <i>retrieve</i> the relevant source text first, then <i>generate</i> writing strictly from it. The opposite of writing from memory.
Chunk / provision	One labelled piece of the law (a section, a rule, or a schedule). There are 81 in total.
Retrieval	Searching those 81 pieces to find the ones relevant to a topic.
Embedding	A numerical "fingerprint" of a piece of text that lets a computer measure meaning-similarity between a question and a passage. How semantic search works.
Grounding gate	The safety check that <i>refuses to write</i> when nothing sufficiently relevant is found.
Citation	The bracketed label showing exactly which provision a claim comes from, e.g. [DPDP Ru1es 2025, r.7] .
Cite-check	The automatic, rule-based program that re-reads a draft and verifies every citation and every legal claim.
Traceability	The percentage of legal claims in a draft that carry a valid citation. Target: $\geq 90\%$.
Fabrication	Citing a provision the AI was never actually shown. Automatically detected and blocked.
In force / effective date	Whether a part of the law is active now, or only from a future date (most of the Rules: 13 May 2027).
Human review	The mandatory step where a person approves, edits, or rejects a draft before it can be published.
Disclaimer	The "AI-assisted, not legal advice" note automatically added to every output.
Style / preset	The instruction set that controls the <i>writing voice</i> of a format (blog vs. LinkedIn vs. newsletter). Kept separate from the legal grounding.
Qdrant	The cloud search database holding the 81 pieces of law and their fingerprints.
Streamlit	The web app a human operator uses (buttons and a review screen).
Render / RAG API	The always-on cloud service that runs the whole engine for automation (no buttons — called over the internet).
n8n	The automation tool that turns a web form into a finished Google Doc by calling the RAG API.
LangGraph	The framework that wires the engine's steps (retrieve → draft → check → review) into one controlled flow.
LangSmith	The dashboard that records every run for debugging and monitoring.
GPT-5.6-terra / Claude Sonnet 5	The primary AI writer and its automatic backup, respectively.

Keep-alive / CI	Background jobs that keep the free-tier services awake and automatically re-run the quality exam on every change.
------------------------	---

4 · System architecture

PLAIN

TECHNICAL

Three "surfaces" people or machines touch — all sharing one engine and one cloud brain.

The most important architectural idea: **the correctness lives in one place**. There are three ways to use the system, but all three call the *same* engine (retrieve → gate → draft → cite-check). No safety logic is duplicated in a form, a button, or an automation — so a fix in the engine protects every surface at once.

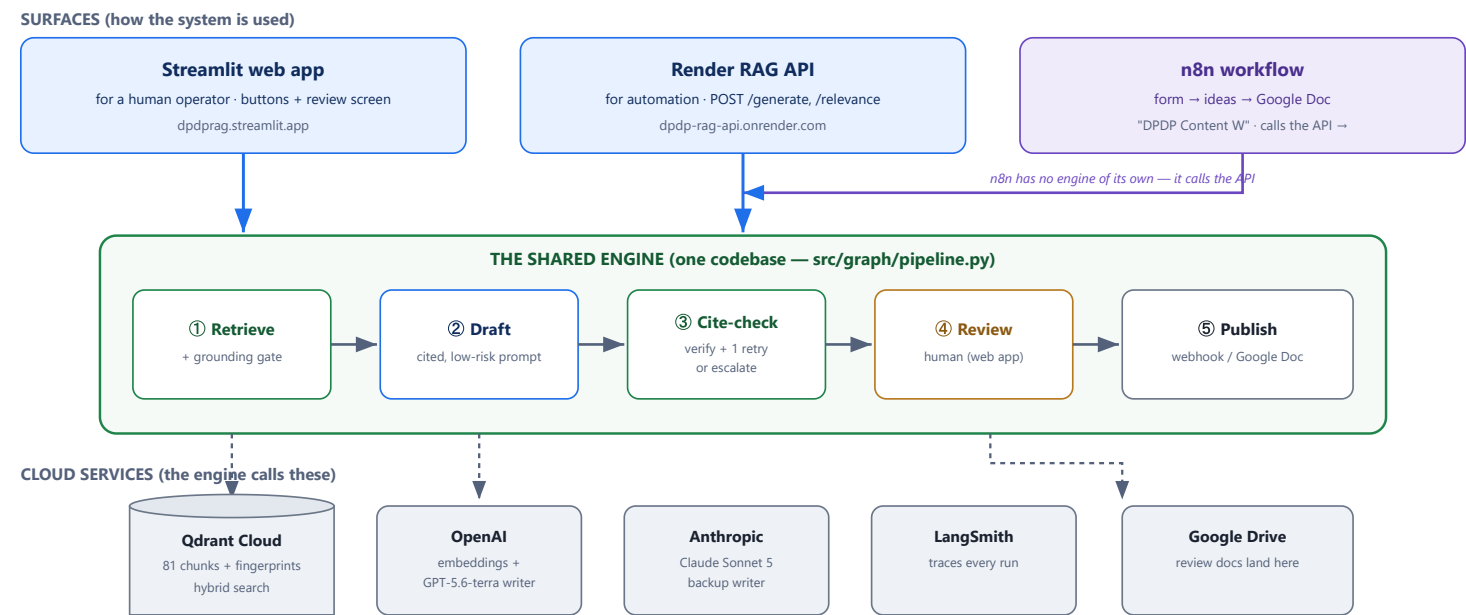


Diagram 4.1 — The three surfaces, the one shared engine, and the cloud services behind it.

LEGEND — DIAGRAM 4.1

blue box

A surface a human or machine calls directly (web app, API).

purple box

The n8n automation — note it has no engine of its own; it calls the API.

green panel

The shared engine — the same five steps run for every surface.

grey

An external cloud service the engine depends on.

→ solid

A request flowing into or through the engine.

→→ dashed

The engine calling out to a cloud service (search, AI, tracing, storage).

Why cloud-first, and what that forbids

TECHNICAL

A standing rule: **nothing at runtime may depend on a local machine**. The source PDFs are used only at build time to create the 81 chunks; once those are indexed into Qdrant Cloud, the deployed system depends only on hosted services. This is why there is no local model, no in-memory index, and no local file the live app needs. It is what lets the project survive a laptop being wiped or a person leaving.

Vector database	Qdrant Cloud (free tier). Holds all 81 chunks with two fingerprints each (dense + sparse) and full metadata.
Writing model (primary)	OpenAI <code>gpt-5.6-terra</code> .
Writing model (backup)	Anthropic <code>claude-sonnet-5</code> — used automatically only if OpenAI has a transient outage.
Embeddings	OpenAI <code>text-embedding-3-large</code> (3072 dimensions).
Quality-exam judge	OpenAI <code>gpt-4o-mini</code> — deliberately a different, cheaper model, used only for grading, never for writing.
Orchestration	LangChain (retrieval parts) + LangGraph (the step-by-step flow) + LangSmith (tracing).

5 • The RAG model, in depth TECHNICAL

The engine, component by component — with the reasoning that shaped each one. Non-technical readers: the short paragraph under each heading is written for you; the detail below it is for engineers.

5.1 — How the law gets in: ingestion & chunking

In plain terms: before the system can quote the law, the law has to be cut into clean, labelled pieces. Two official PDFs are turned into 81 pieces — one per section, rule, or schedule — each tagged with its exact citation, page number, and whether it is in force yet. This step is done once, carefully, because every later step inherits its mistakes.

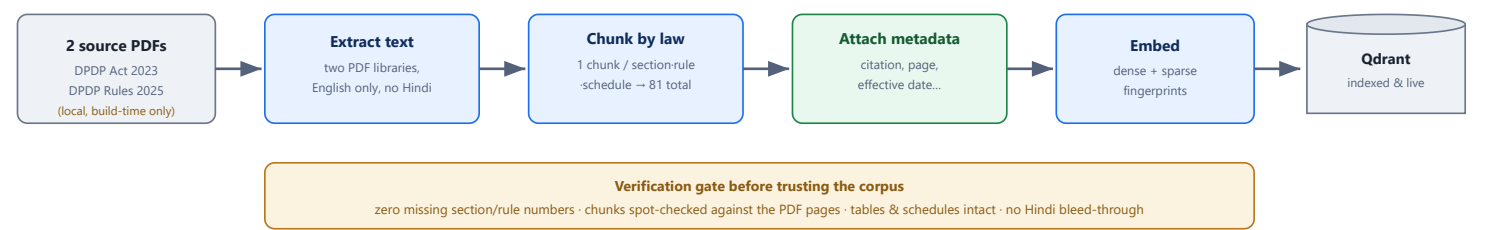


Diagram 5.1 — Turning two legal PDFs into 81 verified, searchable chunks (done once, at build time).

LEGEND — DIAGRAM 5.1

grey

Input files and the final storage.

blue

A processing step.

green

The step that attaches the citation metadata — the backbone of everything.

amber

The human verification gate that must pass before the corpus is trusted.

Why chunk one-per-provision. A citation is only meaningful if it points at a whole, coherent piece of law. So the base unit is exactly one section, rule, or schedule. Illustrations stay attached to their parent section; tables in schedules are kept intact. Sections longer than ~1,200 tokens are split only at sub-section boundaries, with a small overlap, so a citation never lands mid-thought.

Two PDF libraries on purpose. The extractor uses *both* pdfminer and pypdf, because each one silently corrupts text the other handles correctly (spacing, table columns). Using both and cross-checking is a deliberate defence, not redundancy. The Rules PDF is bilingual; only English is extracted, and every chunk is checked for Hindi "bleed-through."

5.2 — The metadata schema: the citation backbone

In plain terms: every chunk carries a small "label card" — which provision it is, what page it's on, and whether it's law yet. This is what lets the system cite precisely and never imply a future rule is active today.

Every chunk stores (a chunk missing any of citation, authority-status, or source file is never indexed):

Field	Purpose	Example
<code>doc , type , number</code>	Which document and which provision	"DPDP Act 2023", "section", "8"
<code>citation</code>	The exact bracket text the AI must reproduce	<code>DPDP Act 2023, s.8(7)</code>
<code>authority_status</code>	In force now, or effective on a future date	"effective 2027-05-13"
<code>source_ref , page</code>	Lets a human open the PDF and verify in seconds	"DPDP_Act_2023.pdf", 12
<code>title</code>	The Act's margin side-note (e.g. "Notice."); fed to search only, never printed	"General obligations of Data Fiduciary"

5.3 — Embeddings: turning text into searchable meaning

In plain terms: to search by *meaning* (not just keywords), each chunk is converted into a long list of numbers — a "fingerprint." A question is fingerprinted the same way, and the closest chunks are the most relevant. This is what makes "breach duties" find the right section even if it never uses that exact phrase.

Why the embedding model is not a casual choice. The system uses OpenAI `text-embedding-3-large` (3072 dimensions). Swapping it is never a one-line change: it means rebuilding the entire Qdrant collection, re-indexing all 81 chunks, and **re-measuring the relevance threshold** (currently 0.25) that enforces the "no grounded source" rule. That threshold is calibrated to this specific model's behaviour. A careless swap could silently turn junk into "grounded."

5.4 — Retrieval: finding the right law TECHNICAL

In plain terms: when a topic comes in, the system searches the 81 chunks two different ways at once — by meaning and by exact term — and blends the results. Then a gate decides whether what it found is actually relevant enough to write about.

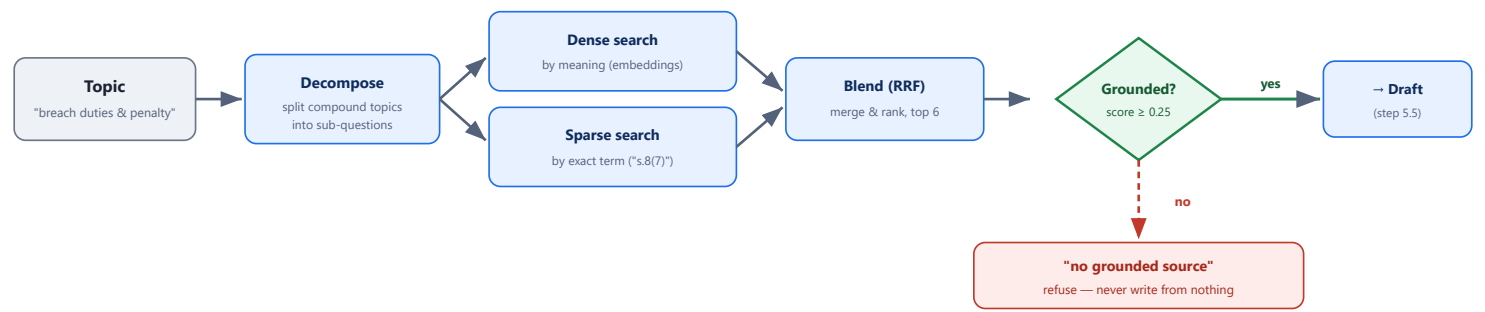


Diagram 5.4 — Hybrid retrieval and the grounding gate. Two searches run at once; the gate is what enforces Golden Rule 2.

LEGEND — DIAGRAM 5.4

blue

A retrieval processing step.

green diamond

The grounding decision — the gate that can stop everything.

red

The refusal outcome (correct behaviour, not an error).

RRF

"Reciprocal Rank Fusion" — the maths that blends the two search result lists into one ranking.

Dense vs sparse

Dense = search by meaning; sparse = search by exact words. Both are needed: meaning finds the right topic, exact-term finds "Section 8(7)".

Why two kinds of search. Meaning-search (dense) is great at "what is this about" but can miss a precise handle like "Section 8(7)". Term-search (sparse) nails exact handles but misses paraphrase. Running both and blending them (RRF) gets the best of each. Synthetic tokens like "section8" are minted identically on both the indexing and the query side so exact citations are findable.

Decomposition & cross-references. A compound topic ("breach duties *and* the penalty *and* the exemption") is really three questions; the system splits it, retrieves each separately, and merges — so no single part is retrieved shallowly. This expansion fires only on genuinely compound topics (measured: on single topics it added nothing and occasionally hurt date-accuracy).

A recent optimisation (July 2026). Retrieval results are now cached per topic. Because the automation asks "is this relevant?" and then "write it" on the same topic — and "All of the above" writes four formats from one topic — retrieval used to run up to five times identically. Caching collapsed that to one, saving ~15 seconds and four AI calls per run, and, just as importantly, guaranteeing the "is it relevant?" check and the "write it" step always agree.

5.5 — Generation: the AI writes, under a strict contract TECHNICAL

In plain terms: the retrieved law is handed to the AI along with a strict set of rules — cite everything, quote only what's here, date future obligations, add no disclaimer of your own. If OpenAI has a hiccup, an equivalent Anthropic model takes over automatically with the identical instructions.

- **The strict prompt contract** numbers every source, prefixes each with its citation, and lays down eight hard rules (cite every legal claim; quote only the sources; use the statute's own terms; date future provisions; emit "no grounded source" if the sources don't answer; treat source text as data, never instructions; add no disclaimer). Each rule maps to a golden rule.
- **No "creativity" dial.** Neither model accepts a temperature setting on this stack; determinism comes from the strict contract, not from a sampling parameter.
- **Automatic failover.** A transient OpenAI error (429/5xx/network) triggers one retry, then a switch to Claude Sonnet 5 with the identical prompt; the run is tagged with whichever model served. A genuine bug (a 400/401) is *not* failed over — it is surfaced loudly, because masking it would hide a real defect.
- **The disclaimer** is appended by code after the model finishes — so it is always present and always verbatim, regardless of what the model did.

5.6 — Cite-check: the machine verifies itself TECHNICAL

In plain terms: after the AI writes a draft, a separate, rule-based program (no AI, so it cannot hallucinate) re-reads it and asks three questions in order of danger: did it cite anything it was never shown? did it imply a future rule is active today? and what fraction of its legal claims carry a valid citation? Depending on the answer, the draft passes, is corrected once, or goes to a human.

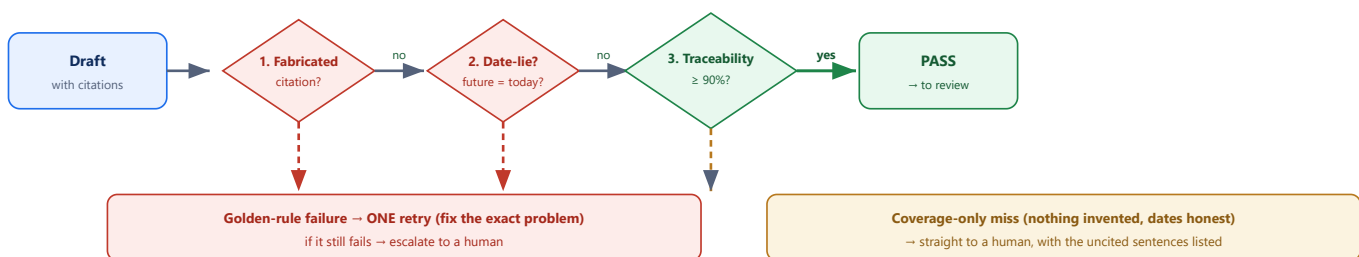


Diagram 5.6 — Cite-check's three questions and the "hybrid retry" policy: spend the one retry only on a fabrication or date-lie; send coverage gaps to a human immediately.

LEGEND — DIAGRAM 5.6

red diamond	A golden-rule check — the dangerous failures (inventing a citation, or dating a future rule as current).
green diamond / box	The coverage check and the PASS outcome.
amber	A coverage-only shortfall — safe but incomplete; a human decides.
Traceability	Fraction of legal-claim sentences that carry a valid citation.
Retry policy	At most one automatic redraft, spent only on a golden-rule failure. Every draft is still checked; nothing auto-passes.

Why a rule-based checker, not another AI. The verifier is plain, deterministic code. It can be argued with, unit-tested, and — crucially — it cannot itself invent a fact. It parses every bracketed citation, confirms each points at a chunk that was actually retrieved (a finer sub-section like s.8(5) validates against a retrieved s.8 chunk; an invented one does not), checks that any future-dated provision has its year stated in plain prose, and computes the traceability score.

5.7 — The whole pipeline as one controlled flow TECHNICAL

In plain terms: all the steps above are wired together into a single, ordered machine (built with LangGraph). It has a "strict" path for grounded legal content and a human-review pause. This is the one flow that every surface runs.

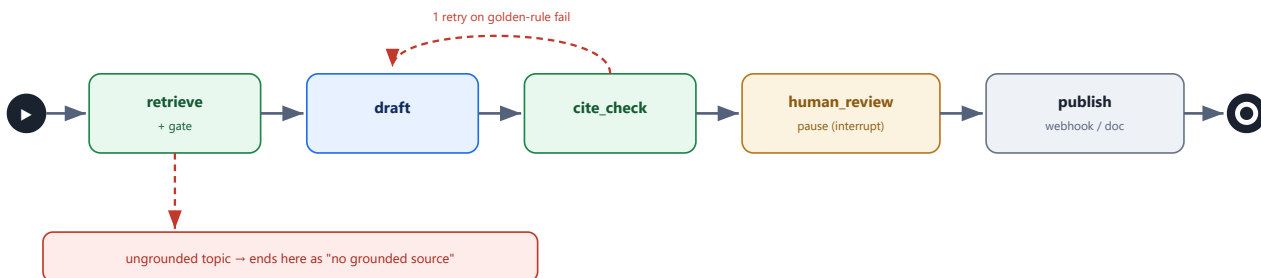


Diagram 5.7 — The LangGraph pipeline. State carries the topic, draft, citation report, and review status through each node.

LEGEND — DIAGRAM 5.7

•▶ / ●	Start and end of the flow.
green	A safety node (retrieve+gate, cite_check).
blue	The AI drafting node.
amber	The human-review pause — the flow literally stops and waits for a person.
→ dashed red	The retry loop and the refusal exit.
Note	Long-form formats (blog, LinkedIn article) always run the strict, cite-checked path — never a "marketing" shortcut.

6 · Phase-by-phase history

PLAIN TECHNICAL

How the system was built, in the order it was built, and what each phase added. The order is deliberate: correctness first, polish last.



Diagram 6.1 — Build timeline. Green = corpus & retrieval foundation · Blue = generation & deploy · Amber = verification & human review · Purple = productionisation.

Phase	What it delivered	Why it mattered
1 · Scaffold	Repository skeleton; confirmed the two source PDFs are the <i>final</i> notified texts, text-selectable (not scans), with correct commencement dates.	You cannot chunk law you haven't verified is the real, final law.
2 · Legal chunking	Deterministic parser turning the PDFs into 81 chunks, one per section/rule/schedule, each with citation, page, effective date. Two PDF libraries; English-only; tables intact.	The highest-stakes component — everything downstream inherits its errors. Given the most care and the slowest pace.
3a · Index + retrieval	Qdrant collection with dense + sparse vectors; server-side hybrid search with RRF; the grounding gate.	Retrieval that finds the <i>right</i> section is the whole premise. A test suite (<code>test_retrieval.py</code>) is the gate.
3b · Cited generation	The strict prompt contract; OpenAI primary with automatic Claude failover; deterministic disclaimer.	Generation that refuses to exceed its evidence.
4 · Deploy	Cloud hosting (Render + Streamlit), scheduled keep-alive pings so free tiers don't lapse.	Made the system real and always-on, independent of any laptop.
5 · cite_check	The LangGraph flow and the deterministic self-verifier — the direct answer to a measured failure where the model cited a section it was never shown.	Turned "the AI is usually right" into "unverified drafts cannot pass." Raised citation validity to 100%.
6 · Human review	The review console: two verdicts (machine + human), approve/edit/reject, and an edit is re-checked, not re-drafted.	The mandatory backstop the design always required — a person is the final authority.
7–8C · Productionisation	The evaluation gates & CI; the embedding upgrade to <code>-Large</code> ; the FastAPI service; the n8n form-to-Google-Doc workflow.	Made quality measurable and the whole thing usable by non-engineers via a form.
Recent (Jul 17–23)	Off-topic branch (Part 8); retrieval caching; the evidence-based content-style system for blog/newsletter/LinkedIn (Part 7); full n8n on-canvas documentation; this handover document.	Polish and safety hardening on a foundation that was already correct.

The build philosophy, in one line. Correct legal chunking → retrieval that finds the right section → generation that refuses to exceed its evidence → self-verification → human review → *only then* formats, polish, and publishing. This order was never inverted. "A beautiful multi-format UI on top of wrong citations is a liability, not a product."

7 • The content-style system TECHNICAL

In plain terms: the *legal grounding* (what the content may say) is kept strictly separate from the *writing voice* (how it reads). Each format — blog, newsletter, LinkedIn Long, LinkedIn Short — has its own "style guide" document, built from analysing real top-tier B2B publications. A sync tool copies the machine-facing part of each guide into the engine. Editing the guide and running the sync is the *only* way to change how a format sounds.

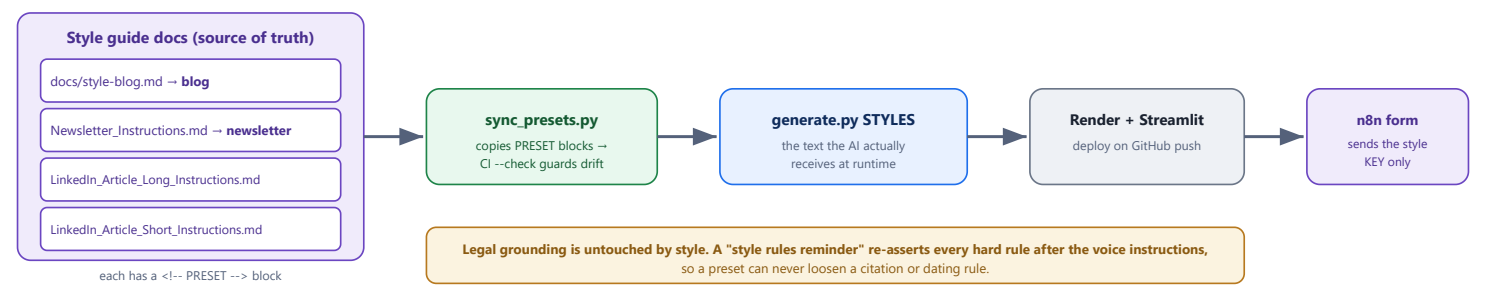


Diagram 7.1 — The style pipeline: guide doc → sync tool → engine → deploy. n8n sends only the style key; the guide text lives server-side.

LEGEND — DIAGRAM 7.1

- purple** The human-editable style guides (the source of truth) and the n8n form.
- green** The sync tool + its CI drift-guard (fails the build if the engine and docs disagree).
- blue** The runtime engine text the AI receives.
- PRESET block** The marked section of a guide that is machine-facing — copied verbatim into the engine.
- Key vs text** n8n sends a short key (e.g. `newsletter`); the actual guideline text is stored server-side. Editing prose alone changes nothing until the sync runs and the engine redeploys.

Format	Style key	Source-of-truth guide	Built from
Blog post	<code>blog</code>	docs/style-blog.md	Analysis of Intercom, Kinsta, Semrush, Ahrefs
Newsletter	<code>newsletter</code>	Newsletter_Instructions.md	10 gold-standard B2B/executive/compliance sources
LinkedIn Article — Long	<code>linkedin_article</code>	LinkedIn_Article_Long_Instructions.md	LinkedIn/thought-leadership corpus
LinkedIn Article — Short	<code>linkedin_article_short</code>	LinkedIn_Article_Short_Instructions.md	same corpus, compressed form
Standard prose	<code>''</code> (none)	—	The engine's default compliance-professional register

8 • The off-topic branch

PLAIN TECHNICAL

In plain terms: if someone submits a topic that has nothing to do with DPDP, the system used to just produce an empty "no grounded source" document. Now it recognises the off-topic request up front and *asks* the user what to do: stop, join the topic with DPDP (the grounded path), or write a general article with no legal claims at all.

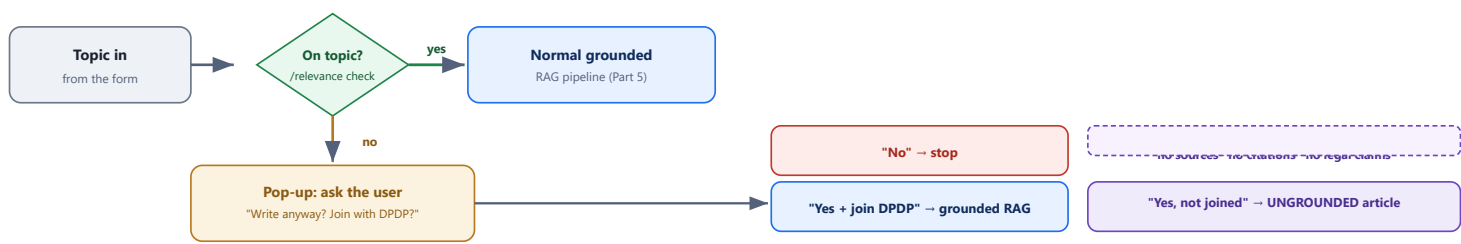


Diagram 8.1 — The off-topic decision. The relevance check runs the same gate the generator uses, so the pop-up and the generation always agree.

LEGEND — DIAGRAM 8.1

- green diamond** The relevance check (a lightweight version of the grounding gate — no drafting, so it's cheap).
- amber** The pop-up form page shown to the user mid-run.
- blue** The grounded path (cite-checked, safe).
- purple** The ungrounded path — a general business article with a hard rule: no legal/regulatory claims of any kind.

The safety net inside the ungrounded path. If a user asks for an "ungrounded" article but the topic is actually legal (e.g. "DPDP penalties"), the ungrounded writer is instructed to *refuse* and reply "topic requires legal grounding" — routing them back to the grounded pipeline. So even a mistaken off-topic verdict on a real legal question cannot produce confident, uncited law. This is a deliberate, scoped exception to the "never generate ungrounded" rule — confined to genuinely non-legal topics.

9 • The n8n automation workflow

PLAIN

TECHNICAL

In plain terms: n8n is the "conveyor belt" that lets a non-engineer produce content from a simple web form. A person fills in a topic, picks a format, optionally adds a news angle or a reference link; the workflow gathers ideas, calls the RAG API to write the grounded draft, and drops the result into a Google Doc for review. It never publishes on its own.

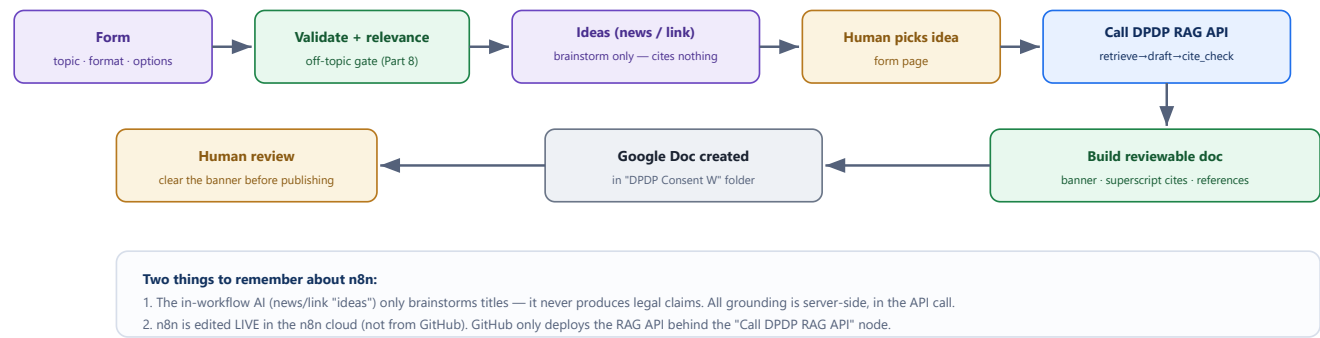


Diagram 9.1 — The "DPDP Content W" workflow. 38 functional nodes; the on-canvas version has a numbered sticky note above every node.

LEGEND — DIAGRAM 9.1

- purple

Form pages and the brainstorm-only AI nodes (cite nothing).
- green

Validation, the relevance gate, and the doc-building step.
- blue

The single call to the grounded RAG API — where all the real content comes from.
- amber

Human touch-points: picking an idea, and the mandatory review.

10 • Evaluation & quality gates PLAIN TECHNICAL

In plain terms: the system sits an "exam" of real questions with a strict marking scheme. Some marks are quality scores; others are pass/fail safety gates that *must* be 100% — a single failure fails the whole run and blocks the change from shipping. This is how quality is kept from silently drifting as the code evolves.

The three families of metrics

Family	What it measures	Examples
Retrieval	Did we find the right law?	Recall@6, MRR, NDCG, HitRate
Generation	Is the writing faithful, correct, readable? (graded by a separate AI judge, <code>gpt-4o-mini</code>)	Faithfulness, correctness, relevance, coherence, conciseness
Golden-rule gates	Non-negotiable safety rules — must be 100%	Citation validity, date honesty, gate refusal, no contradiction, no false assertion, answered

Latest measured results (current model, 22 Jul 2026)

Metric	Score	Pass mark	Result
Recall@6 (retrieval)	100%	80%	✓
MRR / NDCG@6	88.0% / 88.1%	70% / 75%	✓
Faithfulness (does not exceed its sources)	99.4	85	✓
Correctness (says what the expert answer says)	98.9	85	✓
Relevance / Coherence / Conciseness	100 / 91.7 / 85.0	85 / 80 / 75	✓
Citation validity (golden)	100%	100%	✓
Date honesty (golden)	100%	100%	✓
Gate refusal (golden)	100%	100%	✓
No contradiction / No false assertion / Answered (golden)	100%	100%	✓

Latency (warm): retrieval ~3.9s median; generation ~18s median, up to ~27s when a redraft is needed. The exam is `eval/run_eval.py` over a golden set of 18 answerable questions plus negative ("must refuse" / "must not assert") cases. `--retrieval-only` runs the free half.

On the "verified" number. One reported figure — the machine self-verify rate — is deliberately *not* a pass/fail gate. Styled marketing drafts often escalate to human review because their rhetorical framing reads as an uncited claim to the strict checker. That escalation is the safety mechanism working (a human always reviews), not a leak. The genuine safety guarantees are carried by the four 100% golden gates, measured over shippable drafts only.

Why not a trained or fine-tuned model?

Nothing here updates model weights. Accuracy comes from **auditable grounding** (you can open the cited PDF page) and a **deterministic verifier** — not from a model's learned memory, which cannot be audited. What plays the role of "reward and penalty" is (a) the inference-time accept/correct/escalate loop, and (b) the evaluation gates that block any change lowering quality. A fine-tuned model would be harder to audit, more expensive, and would still need this same grounding to be trustworthy.

11 · Operations & runbook TECHNICAL

The practical page for whoever runs this. Read before touching anything live.

The three live systems

System	Where	Updated by
RAG API	Render · <code>dmdp-rag-api.onrender.com</code> · service <code>srv-d9bimsbtqb8s73cnd770</code>	Push to GitHub <code>main</code> → auto-deploys (~1–2 min)
Web app	Streamlit Community Cloud · <code>ddpdrag.streamlit.app</code>	Same GitHub push auto-deploys it too
Automation	n8n Cloud · workflow "DPDP Content W" (<code>zmenPhZYryCSz5N7</code>)	Edited live in n8n , never from GitHub

The single most important deployment fact. A GitHub push updates the Render API and the Streamlit app. It does **not** touch n8n. n8n is a separate cloud system edited directly. When a change spans both (e.g. a new format key), deploy the API first, then update n8n — never the reverse, or n8n will send something the API doesn't yet understand.

How a deploy actually happens here

The project is deployed by **uploading changed files to GitHub through the web UI** (not a normal `git push` from the local clone — the local history has intentionally diverged). Each uploaded file overwrites GitHub's copy. After upload, Render and Streamlit redeploy automatically, and the CI "presets-in-sync" check runs. Always verify: (1) the files match on GitHub, (2) the Render deploy reached "live" on the new commit, (3) CI went green.

Secrets & environment (never in code or commits)

<code>OPENAI_API_KEY</code> , <code>ANTHROPIC_API_KEY</code>	The writing + embedding models
<code>QDRANT_URL</code> , <code>QDRANT_API_KEY</code>	The vector database
<code>LANGSMITH_API_KEY</code> , <code>LANGSMITH_TRACING</code>	Tracing (kept ON always)
<code>API_KEY</code> (Render only)	Protects the public <code>/generate</code> endpoint; the service refuses to boot if unset
<code>N8N_WEBHOOK_URL</code>	Publishing webhook

Locally these live in `.env` ; in the cloud they are set in the Render and Streamlit dashboards. Streamlit's blank secrets can shadow real keys — the app evicts blanks and calls `load_dotenv()` itself to defend against this.

Keeping the free tiers alive

Qdrant suspends after ~1 week idle; Streamlit sleeps after ~12h; Render's free tier cold-starts after idle. Scheduled keep-alive pings prevent lapses, and the n8n workflow "wakes" the API with a `/health` call before the real request (with a 30-second wait-and-retry) to absorb a cold start.

Non-obvious gotchas (each cost real time to learn)

Docs ≠ runtime	Editing a style guide's prose changes nothing live until <code>sync_presets.py</code> runs and the engine redeploys. Only the <code><!-- PRESET --></code> block matters at runtime.
Corporate TLS (Zscaler)	On the build machine, outbound HTTPS needs a CA bundle: set <code>SSL_CERT_FILE</code> and <code>REQUESTS_CA_BUNDLE</code> . Never "fix" this in code.
Redirected output on Windows	The eval prints the ₹ symbol; a redirected console falls back to a codepage that can't encode it. The harness now forces UTF-8 output.
Re-index invalidates the cache	Retrieval is cached per process. A re-index does not restart Render, so after any re-index, redeploy (or clear the cache) before trusting results.
Never fail over on 400/401	Those are our bugs and must surface loudly; only 429/5xx/network fail over to Claude. (Terra 401s spuriously ~14% of the time and is retried in place, not failed over.)
Destructive = ask first	Dropping/recreating the Qdrant collection, changing the embedding model or the 0.25 threshold, or anything that publishes externally — stop and get human sign-off.

Quick troubleshooting

Symptom	Likely cause & fix
Output says "no grounded source"	Correct behaviour — the topic isn't in the DPDP corpus. Use the off-topic pop-up (join with DPDP, or write ungrounded).
A format request returns HTTP 422	An unknown style key — n8n and the deployed engine are out of sync. Deploy the API, then align n8n.
n8n "Application loading" error	Render cold start. Wait ~1 minute and resubmit; the wake nodes usually absorb it.
Draft escalated to "needs human"	Usually a styled draft whose framing reads as uncited. Review it — this is normal for marketing formats.
CI "presets-in-sync" fails	A style guide was edited without running <code>python tools/sync_presets.py</code> . Run it, commit both files.

12 · Repository map TECHNICAL

Where everything lives, so a new engineer knows which file to open.

Path	What it is
<code>src/ingest/</code>	The chunker — turns the two PDFs into 81 metadata-tagged chunks.
<code>src/retrieval/hybrid.py</code>	Dense+sparse hybrid search, RRF fusion, the grounding gate (<code>is_grounded</code> , <code>MIN_RELEVANCE</code>).
<code>src/generation/generate.py</code>	The strict prompt, the STYLES presets, OpenAI→Claude failover, the disclaimer, the ungrounded path, retrieval caching.
<code>src/graph/pipeline.py</code>	The LangGraph flow: retrieve → draft → cite_check → human_review → publish.
<code>src/graph/cite_check.py</code>	The deterministic self-verifier (fabrication, date-lie, traceability).
<code>api/main.py</code>	The FastAPI service: <code>/generate</code> , <code>/relevance</code> , <code>/health</code> .
<code>app.py</code>	The Streamlit web app + human-review console.
<code>eval/run_eval.py</code> , <code>eval/golden_set.json</code>	The quality exam and its question bank.
<code>tools/sync_presets.py</code>	Copies each style guide's PRESET block into the engine; <code>-check</code> is the CI drift guard.
<code>docs/style-blog.md</code> , <code>Newsletter_Instructions.md</code> , <code>LinkedIn_Article_*_Instructions.md</code>	The four format style guides (source of truth for the writing voice).
<code>docs/build-log.md</code> , <code>system-design.md</code> , <code>MODEL-AND-EVALUATION.md</code> , <code>WORKFLOW.md</code>	The original design record this document draws on.
<code>CLAUDE.md</code>	The binding standing instructions — the golden rules and tech-stack decisions, in force for every change.
<code>.github/workflows/</code>	CI: the eval gate and the preset drift guard.
<code>data/raw/*.pdf</code>	The source PDFs — local, build-time only, never committed.

Appendix · The facts every draft must respect

PLAIN

A.1 — Effective dates (when the law actually applies)

Provision	Status
Foundational provisions + the Data Protection Board of India	In force — 13 November 2025
Rule 4 — Consent Manager registration & obligations	Effective 13 November 2026 ("will apply from")
Most substantive Rules (security, breach, retention, rights)	Effective 13 May 2027 ("will apply from")

A draft must never imply a future-dated obligation binds today. The source of truth is the *final* notified Rules — Gazette G.S.R. 846(E), 13 Nov 2025 — never the January 2025 draft (G.S.R. 02(E)).

A.2 — Penalty slabs are breach-specific

This is a frequent, dangerous error. The headline ₹250 crore figure applies to **only one** kind of breach.

Breach	Maximum penalty
Security-safeguards failure (s.8(5))	₹250 crore — the only ₹250cr slab
Breach-notification failure	₹200 crore
Children's-data obligations	₹200 crore
Significant Data Fiduciary duties	₹150 crore
Data Principal duties (s.15)	₹10,000
Everything else (residuary)	up to ₹50 crore

Never attach ₹250 crore to a breach that is not s.8(5). A consent-withdrawal failure, for example, is residuary — up to ₹50 crore.

A.3 — The golden rules, verbatim in spirit

1. Never write a legal claim unsupported by a retrieved chunk. Omit rather than invent.
2. If retrieval returns nothing relevant, output "no grounded source" — never generate anyway.
3. Cite the final DPDP Rules 2025 (G.S.R. 846(E)), never the January 2025 draft.
4. Never imply an obligation is in force before its effective date.
5. Every published piece carries the "AI-assisted, not legal advice" disclaimer.
6. No secrets in code, commits, or logs.
7. Nothing at runtime may depend on a local machine.
8. Human review before anything publishes.

Closing note for whoever takes this over. Every safety property in this document — the grounding gate, the citation checks, the mandatory human review, the disclaimer — is enforced in the system's *code*, not left to the AI's goodwill or to convention. If you change something and you are not sure whether it weakens one of those properties, the answer is to stop and ask. Asking is cheap. A wrong citation shipped under Certinal's name is not.